

Enterprise Deployment Guide

Sysnex Labs

January 2026

- [Enterprise Deployment Guide](#)
 - [Complete Guide for Enterprise Deployment: On-Premise Installation, SSO Integration, and License Management](#)
 - [Table of Contents](#)
 - [Executive Summary](#)
 - [Deployment Architectures](#)
 - [Prerequisites](#)
 - [On-Premise Installation](#)
 - [SSO Integration](#)
 - [License Management](#)
 - [High Availability Setup](#)
 - [Monitoring and Logging](#)
 - [Backup and Disaster Recovery](#)
 - [Security Hardening](#)
 - [Upgrade and Maintenance](#)
 - [Troubleshooting](#)
 - [Summary](#)

Enterprise Deployment Guide

Complete Guide for Enterprise Deployment: On-Premise Installation, SSO Integration, and License Management

Version: 1.0 **Date:** January 2026 **Target Audience:** IT Administrators, DevOps Engineers, Enterprise Architects
Est. Time: 1-2 days (initial deployment)

Table of Contents

1. [Executive Summary](#)
 2. [Deployment Architectures](#)
 3. [Prerequisites](#)
 4. [On-Premise Installation](#)
 5. [SSO Integration](#)
 6. [License Management](#)
 7. [High Availability Setup](#)
 8. [Monitoring and Logging](#)
 9. [Backup and Disaster Recovery](#)
 10. [Security Hardening](#)
 11. [Upgrade and Maintenance](#)
 12. [Troubleshooting](#)
-

Executive Summary

This guide covers enterprise deployment of **NexSuite** for organizations requiring:

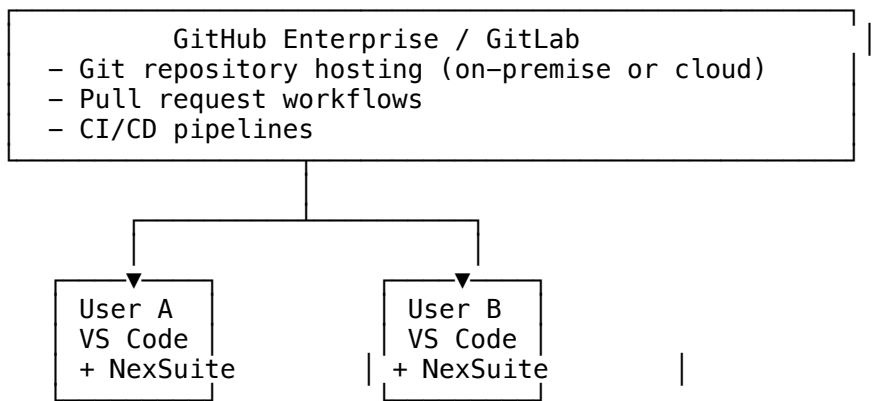
- **On-Premise Deployment:** Air-gapped, data sovereignty, regulatory compliance
- **SSO Integration:** SAML 2.0, OAuth 2.0, LDAP/Active Directory
- **License Management:** Floating licenses, usage tracking, compliance
- **High Availability:** Load balancing, failover, 99.9% uptime SLA
- **Enterprise Security:** TLS/mTLS, secrets management, audit logging

Deployment Options: 1. **VS Code Extension** (Recommended for most enterprises) 2. **SaaS API Server** (On-premise or cloud) 3. **Tauri Desktop App** (Standalone, no VS Code)

Deployment Architectures

Architecture 1: VS Code Extension + Git (Distributed)

Best For: 95% of enterprises (lowest cost, highest flexibility)



Pros:

-  No server infrastructure (lowest cost)
-  Full offline capability (local LSP)
-  Scales to 1000+ users (Git distributed model)
-  Zero single point of failure

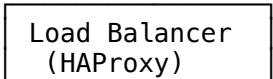
Cons:

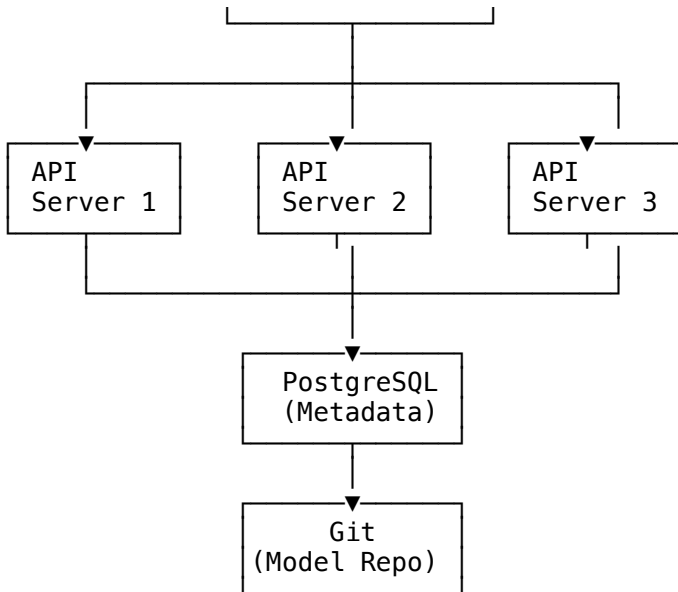
-  Each user installs VS Code extension
-  Requires Git training (minimal)

Infrastructure Cost: \$0-\$21/user/month (GitHub Enterprise)

Architecture 2: SaaS API Server (Centralized)

Best For: Web-based IDE requirements, browser-only environments





Clients: Web browsers (React + Monaco Editor)

- Pros:
- Browser-based (no local install)
 - Centralized license enforcement
 - Audit logging (all requests logged)

- Cons:
- Infrastructure costs (3+ servers)
 - Requires internet connectivity
 - Single point of failure (without HA)

Infrastructure Cost: \$500-\$2,000/month (AWS/Azure, 3-server HA)

Prerequisites

Hardware Requirements (Per API Server Instance)

Component	Minimum	Recommended	Notes
CPU	4 cores	8 cores	Rust benefits from multi-core
RAM	8 GB	16 GB	120MB per user session
Disk	50 GB	200 GB SSD	Git repository + logs
Network	100 Mbps	1 Gbps	For large model sync

Scaling Guidelines: - **Small:** <50 concurrent users → 1 server (8 core, 16 GB) - **Medium:** 50-200 users → 3 servers (8 core, 16 GB each, load balanced) - **Large:** 200-500 users → 5+ servers (16 core, 32 GB each, load balanced)

Software Requirements

Software	Version	Purpose
Operating System	Ubuntu 22.04, RHEL 8+, Windows Server 2019+	Server OS
Docker	24+ (optional)	Containerized deployment
PostgreSQL	14+ (SaaS mode only)	Metadata storage
Git	2.30+	Model repository
Rust	nightly (for source builds)	Compilation
HAProxy / Nginx	Latest (HA mode)	Load balancer

Network Requirements

Firewall Rules:

Port	Protocol	Purpose	Source	Destination
8080	HTTPS	API server	User workstations	API servers
22	SSH	Git access	User workstations	Git server
5432	TCP	PostgreSQL	API servers	PostgreSQL
443	HTTPS	SSO (SAML, OAuth)	API servers	Identity Provider
9090	HTTP	Prometheus metrics	Monitoring	API servers

DNS Requirements: - nexsuite.company.com → Load balancer (SaaS mode) - git.company.com → Git server (if on-premise) - idp.company.com → Identity provider (SSO)

On-Premise Installation

Option 1: VS Code Extension Deployment (Recommended)

Step 1: Package VSIX for Internal Distribution

Download Official VSIX:

```
curl -LO https://github.com/SysnexLabs/nexsuite/releases/download/v0.33.0/nexsuite-sysml-v2-lsp-0.33.0.vsix
```

Host on Internal Server (Optional):

```
# Copy to internal web server
scp nexsuite-sysml-v2-lsp-0.33.0.vsix admin@intranet.company.com:/var/www/downloads/

# Users download from intranet
# https://intranet.company.com/downloads/nexsuite-sysml-v2-lsp-0.33.0.vsix
```

Step 2: Deploy via VS Code Settings Sync (Recommended for 100+ users)

Option A: Use VS Code Settings Sync (GitHub/Microsoft account): 1. Administrator configures reference profile with NexSuite installed 2. Users enable Settings Sync: Ctrl+Shift+P → “Settings Sync: Turn On” 3. Extensions auto-install from profile

Option B: Group Policy (Windows Active Directory):

Create PowerShell deployment script: `deploy-nexsuite.ps1`

```
# Install VS Code (if not installed)
if (!(Test-Path "C:\Program Files\Microsoft VS Code\Code.exe")) {
    Invoke-WebRequest -Uri "https://aka.ms/win32-x64-user-stable" -OutFile "VSCodeSetup.exe"
    Start-Process -FilePath "VSCodeSetup.exe" -Args "/silent /mergetasks=!runcode" -Wait
}

# Install NexSuite extension
$vsixPath = "\\intranet\downloads\nexsuite-sysml-v2-lsp-0.33.0.vsix"
code --install-extension $vsixPath

# Configure settings
$settingsPath = "$env:APPDATA\Code\User\settings.json"
$settings = @"
{
    "sysml.lsp.logLevel": "info",
    "sysml.aspice.enabled": true,
    "sysml.lsp.standardLibrary.path": "\\intranet\sysml\library"
}
"@
Set-Content -Path $settingsPath -Value $settings
```

Deploy via Group Policy: 1. Active Directory → Group Policy Management 2. Create new GPO: “Deploy NexSuite” 3. Computer Configuration → Policies → Windows Settings → Scripts → Startup 4. Add script: \\domain\netlogon\deploy-nexsuite.ps1 5. Link GPO to target OU (e.g., Engineering)

Step 3: License Activation

Enterprise License Server (Floating Licenses):

Install License Server:

```
# Download license server binary
curl -LO https://github.com/SysnexLabs/nexsuite/releases/download/v0.33.0/nexsuite-license-server-linux-amd64

# Copy to /usr/local/bin
sudo cp nexsuite-license-server-linux-amd64 /usr/local/bin/nexsuite-license-server
sudo chmod +x /usr/local/bin/nexsuite-license-server

# Create systemd service
sudo tee /etc/systemd/system/nexsuite-license.service <<EOF
[Unit]
Description=NexSuite License Server
After=network.target

[Service]
Type=simple
User=nexsuite
ExecStart=/usr/local/bin/nexsuite-license-server \
    --port 8090 \
    --license-file /etc/nexsuite/license.key \
    --max-concurrent-users 100
Restart=always

[Install]
```

```
WantedBy=multi-user.target
EOF
```

```
# Start service
sudo systemctl enable nexsuite-license
sudo systemctl start nexsuite-license
```

Configure Clients (VS Code settings.json):

```
{
  "sysml.license.server": "https://license.company.com:8090",
  "sysml.license.checkoutTimeout": 3600 // 1 hour
}
```

License Checkout Flow: 1. User opens VS Code with NexSuite extension 2. Extension requests license from server 3. Server checks available licenses (e.g., 85/100 in use) 4. Server grants license token (1-hour timeout) 5. Extension renews token every 30 minutes (heartbeat) 6. On VS Code close, license returned to pool

Option 2: SaaS API Server Deployment

Step 1: Docker Container Deployment

Pull Official Image:

```
docker pull sysnexplabs/nexsuite-api-server:0.33.0
```

Create Docker Compose Configuration: docker-compose.yml

```
version: '3.8'

services:
  postgres:
    image: postgres:14
    environment:
      POSTGRES_DB: nexsuite
      POSTGRES_USER: nexsuite
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    volumes:
      - postgres_data:/var/lib/postgresql/data
    secrets:
      - db_password
    networks:
      - nexsuite_net

  api-server-1:
    image: sysnexplabs/nexsuite-api-server:0.33.0
    environment:
      DATABASE_URL: postgresql://nexsuite@postgres:5432/nexsuite
      GIT_REPO_PATH: /data/models
      SSO_PROVIDER: saml
      SSO_SAML_IDP_URL: https://idp.company.com/saml/sso
      SSO_SAML_CERT_PATH: /run/secrets/saml_cert
      LOG_LEVEL: info
    volumes:
      - git_models:/data/models
      - logs:/var/log/nexsuite
    secrets:
      - db_password
```

```

    - saml_cert
ports:
    - "8080:8080"
networks:
    - nexsuite_net
depends_on:
    - postgres

api-server-2:
image: sysnexlabs/nexsuite-api-server:0.33.0
environment:
    DATABASE_URL: postgresql://nexsuite@postgres:5432/nexsuite
    GIT_REPO_PATH: /data/models
    SSO_PROVIDER: saml
    SSO_SAML_IDP_URL: https://idp.company.com/saml/sso
    SSO_SAML_CERT_PATH: /run/secrets/saml_cert
    LOG_LEVEL: info
volumes:
    - git_models:/data/models
    - logs:/var/log/nexsuite
secrets:
    - db_password
    - saml_cert
ports:
    - "8081:8080"
networks:
    - nexsuite_net
depends_on:
    - postgres

haproxy:
image: haproxy:2.8
volumes:
    - ./haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro
ports:
    - "443:443"
networks:
    - nexsuite_net
depends_on:
    - api-server-1
    - api-server-2

volumes:
    postgres_data:
    git_models:
    logs:

networks:
    nexsuite_net:

secrets:
    db_password:
        file: ./secrets/db_password.txt
    saml_cert:
        file: ./secrets/saml_cert.pem

```

HAProxy Configuration: haproxy.cfg

```
global
    log /dev/log local0
    maxconn 4096

defaults
    log global
    mode http
    option httplog
    timeout connect 5000ms
    timeout client 50000ms
    timeout server 50000ms

frontend https_frontend
    bind *:443 ssl crt /etc/ssl/certs/nexsuite.pem
    default_backend api_servers

backend api_servers
    balance roundrobin
    option httpchk GET /health
    http-check expect status 200

    server api1 api-server-1:8080 check
    server api2 api-server-2:8080 check
```

Step 2: Launch Stack

```
# Create secrets directory
mkdir -p secrets
echo "your-db-password" > secrets/db_password.txt
cp /path/to/saml-cert.pem secrets/saml_cert.pem

# Launch stack
docker-compose up -d

# Verify health
curl https://nexsuite.company.com/health

# Expected output:
# {"status": "ok", "version": "0.33.0", "uptime": "5m"}
```

SSO Integration

SAML 2.0 Integration (Okta, Azure AD, Google Workspace)

Step 1: Configure Identity Provider (Okta Example)

Okta Admin Console: 1. Applications → Create App Integration 2. Sign-On Method: **SAML 2.0** 3. App Settings: - **Single Sign-On URL:** <https://nexsuite.company.com/auth/saml/callback> - **Audience URI (SP Entity ID):** <https://nexsuite.company.com> - **Attribute Statements:** - email → user.email - firstName → user.firstName - lastName → user.lastName - groups → appuser.groups 4. Download **IdP metadata XML**

Step 2: Configure NexSuite API Server

Environment Variables (.env):


```
SSO_PROVIDER=saml
SSO_SAML_IDP_URL=https://company.okta.com/app/abc123/sso/saml
SSO_SAML_IDP_METADATA=/etc/nexsuite/okta-metadata.xml
SSO_SAML_SP_ENTITY_ID=https://nexsuite.company.com
SSO_SAML_CALLBACK_URL=https://nexsuite.company.com/auth/saml/callback
```

Group-based authorization

```
SSO_SAML_GROUP_ATTRIBUTE=groups
SSO_SAML_ADMIN_GROUP=NexSuite-Admins
SSO_SAML_ENGINEER_GROUP=NexSuite-Engineers
```

Copy IdP Metadata:

```
docker cp okta-metadata.xml nexsuite-api-server-1:/etc/nexsuite/okta-metadata.xml
```

Step 3: Test SSO Login

1. Navigate to <https://nexsuite.company.com>
2. Click “Sign in with SSO”
3. Redirect to Okta login
4. Enter credentials
5. Redirect back to NexSuite with SAML assertion
6. Session established (JWT token issued)

Verify JWT Token:

```
curl -H "Authorization: Bearer $JWT_TOKEN" \
https://nexsuite.company.com/api/user/me
```

Response:

```
# {
#   "email": "alice@company.com",
#   "firstName": "Alice",
#   "lastName": "Engineer",
#   "groups": ["NexSuite-Engineers"],
#   "role": "engineer"
# }
```

OAuth 2.0 / OIDC Integration (GitHub Enterprise, GitLab)

Step 1: Register OAuth App (GitHub Enterprise)

GitHub Enterprise Admin: 1. Settings → Developer settings → OAuth Apps → New OAuth App 2. Application name: **NexSuite** 3. Homepage URL: <https://nexsuite.company.com> 4. Authorization callback URL: <https://nexsuite.company.com/auth/oauth/callback> 5. Note **Client ID** and **Client Secret**

Step 2: Configure NexSuite

Environment Variables (.env):

```
SSO_PROVIDER=oauth2
SSO_OAUTH2_CLIENT_ID=Iv1.abc123def456
SSO_OAUTH2_CLIENT_SECRET_FILE=/run/secrets/oauth_client_secret
SSO_OAUTH2_AUTHORIZE_URL=https://github-enterprise.company.com/login/oauth/authorize
SSO_OAUTH2_TOKEN_URL=https://github-enterprise.company.com/login/oauth/access_token
```

```
SSO_OAUTH2_USER_INFO_URL=https://github-enterprise.company.com/api/v3/user
SSO_OAUTH2_SCOPES=user:email,read:org
```

Docker Secret:

```
echo "your-client-secret" | docker secret create oauth_client_secret -
```

LDAP / Active Directory Integration

Environment Variables (.env):

```
SSO_PROVIDER=ldap
SSO_LDAP_URL=ldaps://ad.company.com:636
SSO_LDAP_BIND_DN=CN=nexsuite-service,OU=ServiceAccounts,DC=company,DC=com
SSO_LDAP_BIND_PASSWORD_FILE=/run/secrets/ldap_password
SSO_LDAP_BASE_DN=DC=company,DC=com
SSO_LDAP_USER_FILTER=(sAMAccountName={username})
SSO_LDAP_GROUP_FILTER=(member={userDN})

# Group mapping
SSO_LDAP_ADMIN_GROUP=CN=NexSuite-Admins,OU=Groups,DC=company,DC=com
SSO_LDAP_ENGINEER_GROUP=CN=Engineers,OU=Groups,DC=company,DC=com
```

License Management

License Types

License Type	Description	Use Case
Node-Locked	Tied to specific machine (MAC address)	Offline workstations, contractors
Floating	Concurrent user pool (e.g., 100 licenses)	Shared team resources
Subscription	Time-based (annual renewal)	SaaS deployments

Floating License Server

Installation (see Step 3 under “VS Code Extension Deployment”)

License File Format (/etc/nexsuite/license.key):

```
-----BEGIN NEXSUIE LICENSE-----
LicenseType: Floating
MaxConcurrentUsers: 100
Tier: Automotive
ExpirationDate: 2027-01-01
Features: ASPICE,ISO26262,UVL,VSS
CompanyName: Acme Corporation
LicenseKey: ABC123...XYZ789
Signature: SHA256-HMAC...
-----END NEXSUIE LICENSE-----
```

Verify License:

```
nexsuite-license-server --verify /etc/nexsuite/license.key
```

```
# Output:
# ✓ License valid
# ✓ Tier: Automotive
# ✓ Max users: 100
# ✓ Expires: 2027-01-01 (363 days remaining)
```

Usage Tracking

Prometheus Metrics (Exposed on :9090/metrics):

```
# HELP nexsuite_license_total Total licenses available
# TYPE nexsuite_license_total gauge
nexsuite_license_total{tier="automotive"} 100

# HELP nexsuite_license_in_use Licenses currently in use
# TYPE nexsuite_license_in_use gauge
nexsuite_license_in_use{tier="automotive"} 87

# HELP nexsuite_license_checkouts_total Total license checkouts
# TYPE nexsuite_license_checkouts_total counter
nexsuite_license_checkouts_total{user="alice@company.com"} 245
```

Query in Prometheus:

```
# Current license utilization percentage
(nexsuite_license_in_use / nexsuite_license_total) * 100

# Example result: 87% utilization
```

License Renewal Workflow

60 Days Before Expiration: - Email alert to admin: “License expires in 60 days” - Dashboard shows warning banner

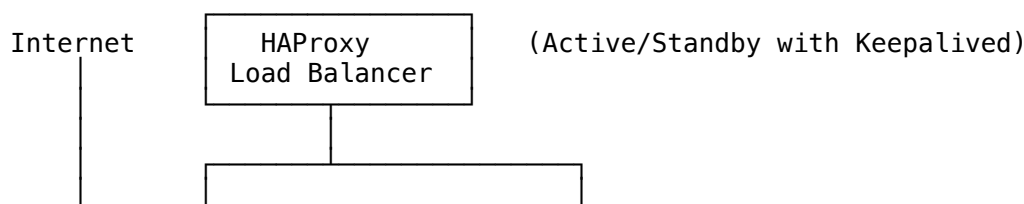
30 Days Before Expiration: - Daily email reminders - VS Code extension shows warning notification

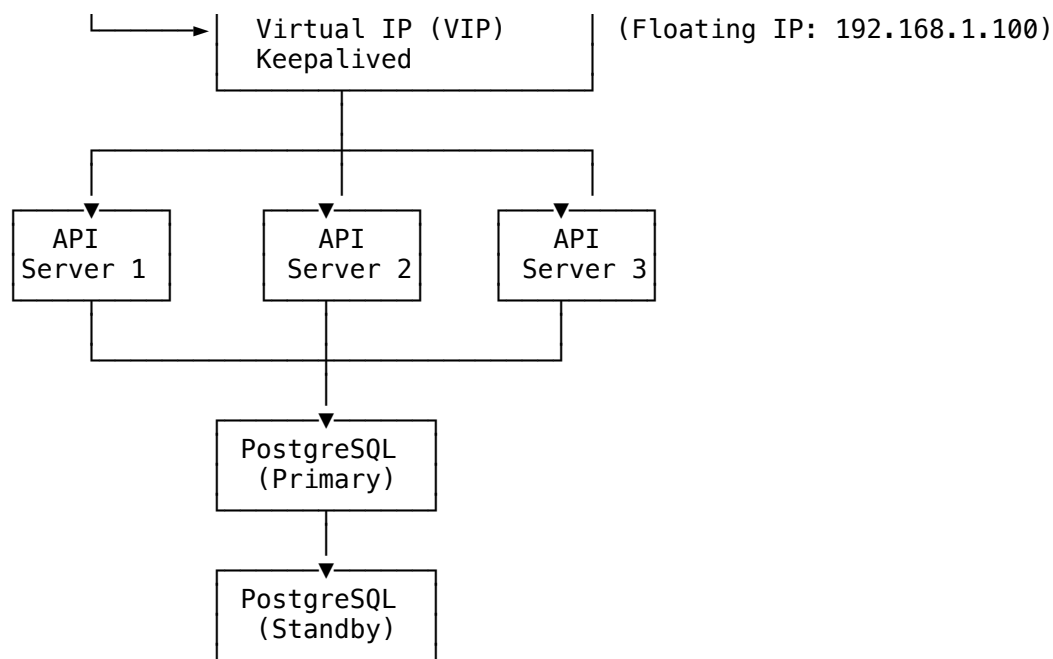
Renewal Process: 1. Contact Sysnex Labs: licensing@sysnex-labs.com 2. Receive new license file 3. Update /etc/nexsuite/license.key 4. Restart license server: `sudo systemctl restart nexsuite-license` 5. No downtime (clients auto-reconnect)

High Availability Setup

3-Server HA Configuration

Architecture:





Keepalived Configuration (/etc/keepalived/keepalived.conf):

```
vrrp_instance VI_1 {
    state MASTER
    interface eth0
    virtual_router_id 51
    priority 100
    advert_int 1

    authentication {
        auth_type PASS
        auth_pass secret123
    }

    virtual_ipaddress {
        192.168.1.100/24
    }
}
```

PostgreSQL Replication (Streaming Replication):

Primary Server (postgresql.conf):

```
wal_level = replica
max_wal_senders = 3
wal_keep_size = 1GB
```

Standby Server (recovery.conf):

```
standby_mode = on
primary_conninfo = 'host=192.168.1.10 port=5432 user=replicator password=secret'
restore_command = 'cp /var/lib/pgsql/archive/%f %p'
```

Automatic Failover: Use repmgr or Patroni for automated failover

Monitoring and Logging

Prometheus + Grafana Stack

Deploy Monitoring Stack (docker-compose.monitoring.yml):

```
version: '3.8'

services:
  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
      - prometheus_data:/prometheus
    ports:
      - "9090:9090"
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.retention.time=30d'

  grafana:
    image: grafana/grafana:latest
    volumes:
      - grafana_data:/var/lib/grafana
      - ./grafana-dashboards:/etc/grafana/provisioning/dashboards
    ports:
      - "3000:3000"
    environment:
      GF_SECURITY_ADMIN_PASSWORD: admin123
      GF_INSTALL_PLUGINS: grafana-clock-panel

volumes:
  prometheus_data:
  grafana_data:
```

Prometheus Configuration (prometheus.yml):

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'nexsuite-api-servers'
    static_configs:
      - targets:
        - 'api-server-1:9090'
        - 'api-server-2:9090'
        - 'api-server-3:9090'

  - job_name: 'nexsuite-license-server'
    static_configs:
      - targets:
        - 'license-server:9090'
```

Key Metrics: - nexsuite_requests_total (total API requests) - nexsuite_request_duration_seconds (p50, p95, p99 latency) - nexsuite_license_in_use (license utilization) - nexsuite_errors_total (error rate)

Centralized Logging (ELK Stack)

Elasticsearch + Logstash + Kibana:

```
# docker-compose.logging.yml
version: '3.8'

services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.11.0
    environment:
      - discovery.type=single-node
    volumes:
      - es_data:/usr/share/elasticsearch/data
    ports:
      - "9200:9200"

  logstash:
    image: docker.elastic.co/logstash/logstash:8.11.0
    volumes:
      - ./logstash.conf:/usr/share/logstash/pipeline/logstash.conf
    ports:
      - "5044:5044"

  kibana:
    image: docker.elastic.co/kibana/kibana:8.11.0
    ports:
      - "5601:5601"
    environment:
      ELASTICSEARCH_HOSTS: http://elasticsearch:9200

volumes:
  es_data:
```

Logstash Configuration (logstash.conf):

```
input {
  file {
    path => "/var/log/nexsuite/*.log"
    type => "nexsuite-api"
  }
}

filter {
  json {
    source => "message"
  }
}

output {
  elasticsearch {
    hosts => ["elasticsearch:9200"]
    index => "nexsuite-%{+YYYY.MM.dd}"
  }
}
```

Backup and Disaster Recovery

Backup Strategy

What to Backup: 1. **Git Repository** (models): Daily full backup 2. **PostgreSQL** (metadata): Hourly incremental, daily full 3. **License Keys**: Weekly (immutable) 4. **Configuration Files**: Daily

Git Repository Backup

Option 1: Git Mirrors

```
# Create bare mirror
git clone --mirror https://git.company.com/models/vehicle.git vehicle-backup.git

# Schedule daily sync (cron)
0 2 * * * cd /backups/vehicle-backup.git && git remote update
```

Option 2: Filesystem Backup (rsync)

```
rsync -avz --delete /data/git-repos/ backup-server:/backups/git-repos/
```

PostgreSQL Backup

Automated Backup Script (backup-postgres.sh):

```
#!/bin/bash
BACKUP_DIR=/backups/postgres
DATE=$(date +%Y%m%d-%H%M)

# Full backup
pg_dump -h postgres -U nexsuite -F c -f $BACKUP_DIR/nexsuite-$DATE.dump nexsuite

# Retention: keep 30 days
find $BACKUP_DIR -name "*.dump" -mtime +30 -delete

# Upload to S3 (optional)
aws s3 cp $BACKUP_DIR/nexsuite-$DATE.dump s3://company-backups/nexsuite/
```

Cron Schedule:

```
0 2 * * * /usr/local/bin/backup-postgres.sh
```

Disaster Recovery Procedure

Scenario: Complete data center failure

RTO (Recovery Time Objective): 4 hours **RPO (Recovery Point Objective):** 24 hours (daily backups)

Recovery Steps:

1. Provision New Infrastructure (1 hour)

- Deploy 3 new servers (cloud or DR site)
- Install Docker + docker-compose

2. Restore Git Repository (1 hour)

```
rsync -avz backup-server:/backups/git-repos/ /data/git-repos/
```

3. Restore PostgreSQL (30 min)

```
pg_restore -h postgres-new -U nexsuite -d nexsuite /backups/nexsuite-20260103.dump
```

4. Deploy NexSuite Stack (30 min)

```
docker-compose up -d
```

5. Verify Health (1 hour)

- Health checks pass
- Test user login (SSO)
- Test LSP operations (sample model)

Total Recovery Time: 4 hours

Security Hardening

TLS/mTLS Configuration

Generate Self-Signed Certificate (Internal CA):

Create CA

```
openssl genrsa -out ca-key.pem 4096
openssl req -new -x509 -days 3650 -key ca-key.pem -out ca-cert.pem
```

Create server certificate

```
openssl genrsa -out server-key.pem 4096
openssl req -new -key server-key.pem -out server-csr.pem -subj "/CN=nexsuite.company.com"
openssl x509 -req -days 365 -in server-csr.pem -CA ca-cert.pem -CAkey ca-key.pem -
CAcreateserial -out server-cert.pem
```

HAProxy TLS Configuration (haproxy.cfg):

```
frontend https_frontend
    bind *:443 ssl crt /etc/ssl/certs/nexsuite.pem verify required ca-file
/etc/ssl/certs/ca-cert.pem
    # ... rest of config
```

mTLS (Client Certificate Authentication): - Clients present certificate signed by company CA - HAProxy verifies client certificate before allowing connection

Secrets Management (HashiCorp Vault)

Store Secrets in Vault:

Initialize Vault

```
vault kv put secret/nexsuite/db password="your-db-password"
vault kv put secret/nexsuite/saml cert=@saml-cert.pem
vault kv put secret/nexsuite/oauth client-secret="your-oauth-secret"
```

Retrieve Secrets in Docker Compose:

```
services:
  api-server-1:
```



```
environment:
  DATABASE_PASSWORD: ${VAULT_SECRET_DB_PASSWORD}
command:
  - sh
  - -c
  - |
    export DATABASE_PASSWORD=$(vault kv get -field=password secret/nexsuite/db)
    /usr/local/bin/nexsuite-api-server
```

Audit Logging

Log All API Requests (JSON format):

```
{
  "timestamp": "2026-01-03T14:23:45Z",
  "user": "alice@company.com",
  "ip": "192.168.1.50",
  "method": "POST",
  "path": "/api/models/vehicle.sysml",
  "status": 200,
  "duration_ms": 45,
  "user_agent": "VS Code/1.85.0"
}
```

Audit Trail Requirements: -  All authentication attempts (success/failure) -  All model modifications (create, update, delete) -  All license checkouts/checkins -  All admin actions (user management, configuration changes)

Retention: 90 days (configurable)

Upgrade and Maintenance

Rolling Upgrade (Zero Downtime)

Step 1: Upgrade API Server 1

```
# Drain server 1 (HAProxy)
echo "disable server api_servers/api1" | socat stdio /var/run/haproxy.sock

# Stop container
docker stop nexsuite-api-server-1

# Pull new image
docker pull sysnexlabs/nexsuite-api-server:0.34.0

# Update docker-compose.yml (change version to 0.34.0)

# Start new container
docker-compose up -d api-server-1

# Verify health
curl http://localhost:8080/health
```

```
# Re-enable in HAProxy
echo "enable server api_servers/api1" | socat stdio /var/run/haproxy.sock
```

Step 2: Repeat for Server 2,3

Total Downtime: 0 seconds (rolling upgrade)

Database Schema Migrations

Automatic Migration (on API server startup):

```
# NexSuite API server runs migrations on startup
# Using diesel-cli or sqlx migrations

# Example migration SQL
-- migrations/2026-01-03-add-audit-log-table.sql
CREATE TABLE audit_log (
    id SERIAL PRIMARY KEY,
    timestamp TIMESTAMPTZ NOT NULL,
    user_email VARCHAR(255) NOT NULL,
    action VARCHAR(50) NOT NULL,
    resource VARCHAR(255),
    details JSONB
);
```

Manual Migration (if needed):

```
docker exec -it nexsuite-api-server-1 diesel migration run
```

Troubleshooting

Common Issues

Issue 1: API Server Not Starting

Symptoms: Container exits immediately

Check Logs:

```
docker logs nexsuite-api-server-1
```

```
# Common errors:
# - "Failed to connect to database": Check DATABASE_URL
# - "License file not found": Check /etc/nexsuite/license.key
# - "SAML metadata invalid": Check SSO_SAML_IDP_METADATA
```

Solutions: 1. Verify environment variables: `docker exec nexsuite-api-server-1 env` 2. Test database connection: `psql -h postgres -U nexsuite -d nexsuite` 3. Validate license: `nexsuite-license-server --verify /etc/nexsuite/license.key`

Issue 2: SSO Login Fails

Symptoms: Redirect loop, “Invalid SAML Response”

Debug Steps: 1. Enable debug logging: LOG_LEVEL=debug 2. Check SAML assertion: `bash docker logs nexsuite-api-server-1 | grep SAML` # Look for "SAML assertion received" with details 3. Verify certificate: `bash openssl x509 -in /etc/nexsuite/saml-cert.pem -text -noout` # Check expiration date, issuer

Issue 3: High License Utilization (100% in use)

Symptoms: Users unable to check out licenses

Investigation:

`curl http://license-server:8090/api/licenses/active`

Response shows all active sessions:

```
# [  
#   {"user": "alice@company.com", "checked_out": "2026-01-03T10:00:00Z", "expires": "2026-  
#     01-03T11:00:00Z"},  
#   {"user": "bob@company.com", "checked_out": "2026-01-03T09:30:00Z", "expires": "2026-01-  
#     03T10:30:00Z"}  
# ]
```

Solutions: 1. **Reduce timeout:** SSO_LICENSE_CHECKOUT_TIMEOUT=1800 (30 min instead of 1 hour) 2. **Reclaim stale licenses:** `curl -X POST http://license-server:8090/api/licenses/reclaim` 3. **Purchase additional licenses:** Contact licensing@sysnex-labs.com

Summary

Deployment Checklist:

-  Choose deployment architecture (VS Code + Git recommended)
-  Install infrastructure (servers, Docker, PostgreSQL)
-  Configure SSO (SAML, OAuth, or LDAP)
-  Set up license server (floating licenses)
-  Configure high availability (3+ servers, load balancer)
-  Set up monitoring (Prometheus, Grafana)
-  Configure backups (Git, PostgreSQL, daily)
-  Harden security (TLS/mTLS, secrets management, audit logging)
-  Test disaster recovery procedure (quarterly)

Support: - **Enterprise Support SLA:** 4-hour response time - **Email:** enterprise@sysnex-labs.com - **Phone:** +1 (555) 123-4567 - **Professional Services:** Available for deployment assistance

Contact Information

- **Website:** <https://sysnexlabs.github.io>
- **Email:** enterprise@sysnex-labs.com

- **Documentation:** <https://docs.nexsuite.dev/enterprise>

NexSuite Enterprise - Secure, Scalable, Compliant MBSE